

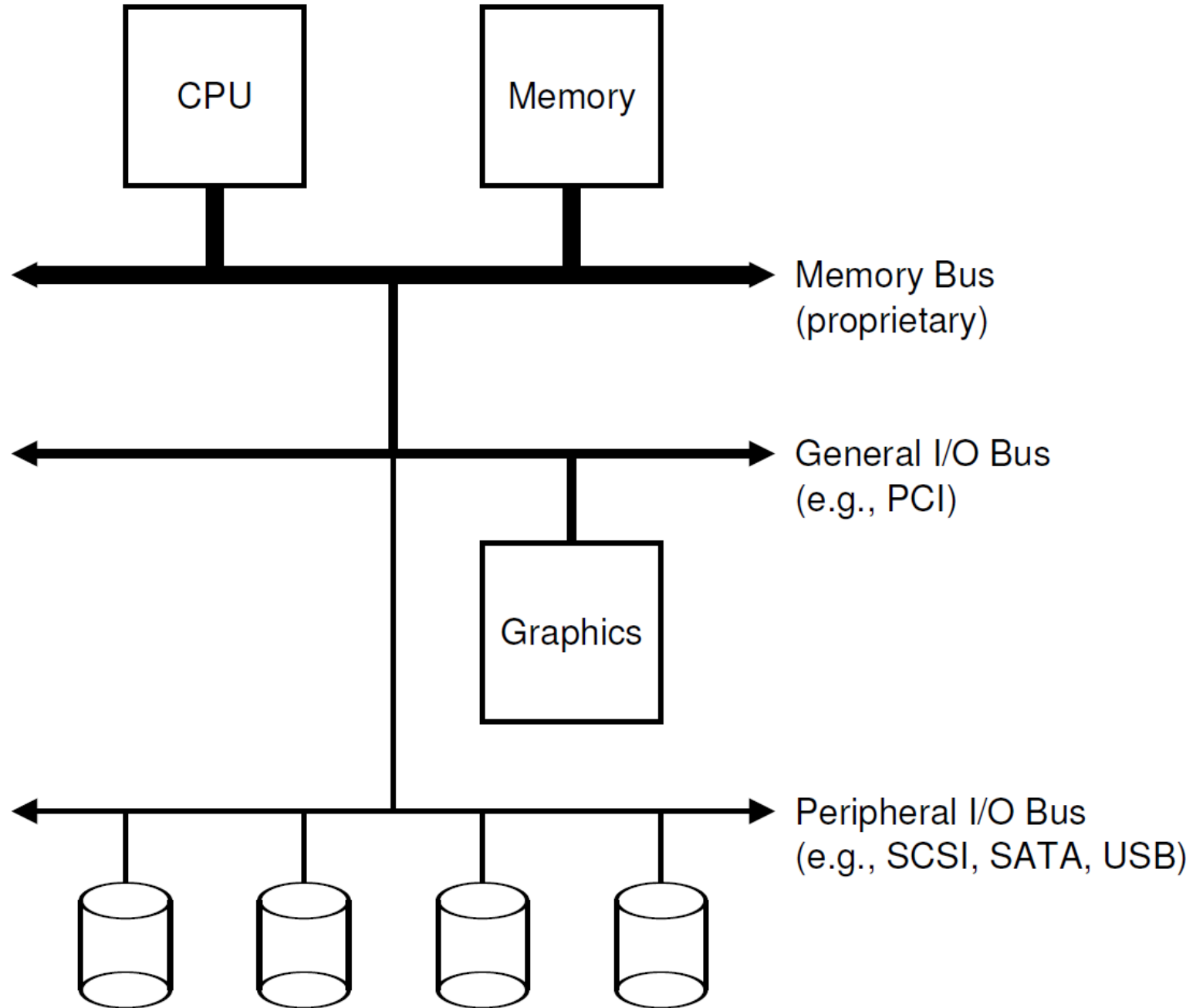
I/O Devices

CS311 Operating System, Fall 2020

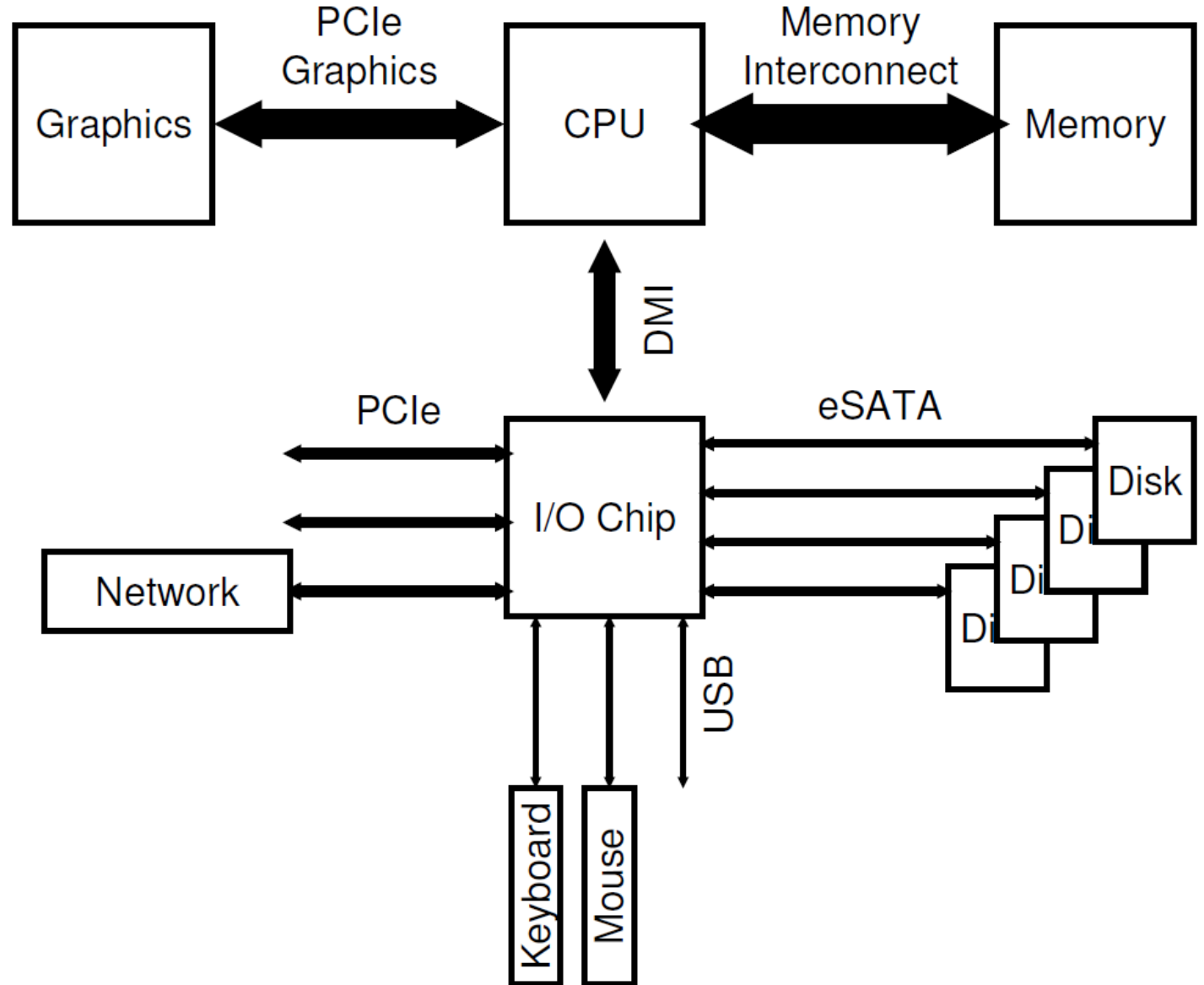
- Imagine a program without any:
 - Input
 - Output
 - Both?

ProtoTypical System

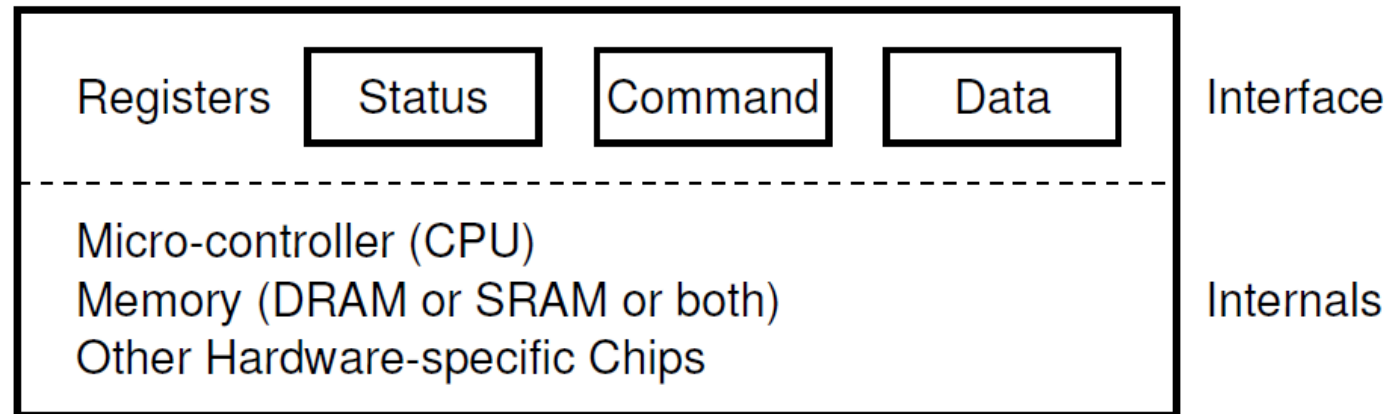
- Why hierarchy?



Typical Modern System



A typical I/O device



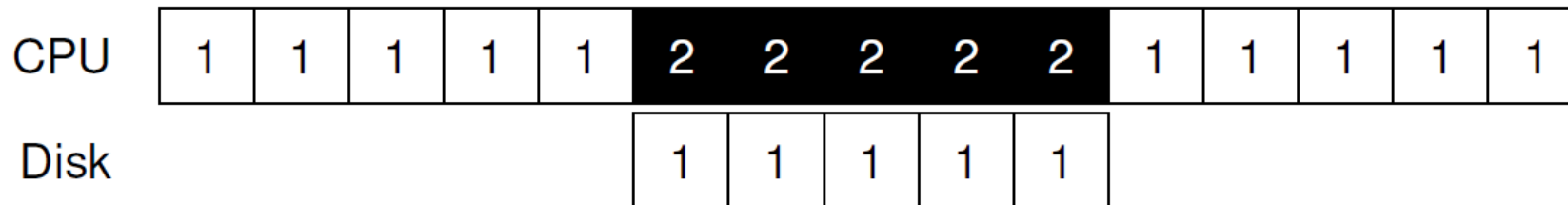
A typical protocol

```
While (STATUS == BUSY)
    ; // wait until device is not busy
Write data to DATA register
Write command to COMMAND register
    (Doing so starts the device and executes the command)
While (STATUS == BUSY)
    ; // wait until device is done with your request
```

- Polling
- Programmed I/O
- Polling is inefficient!

Interrupts

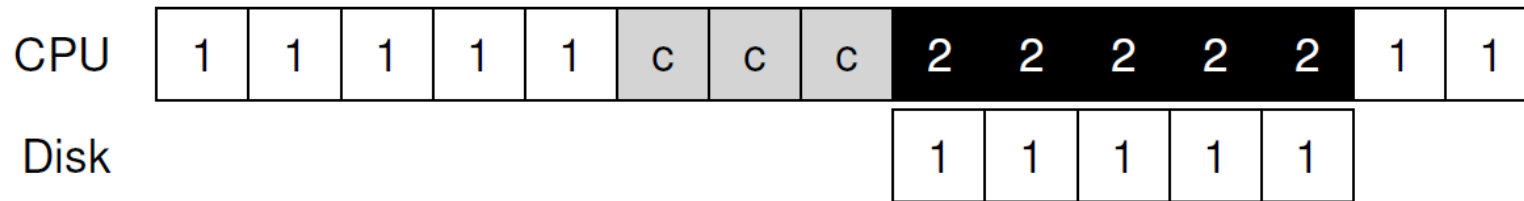
- Let computation and I/O overlap
 - OS writes the data and the command
 - It puts the calling process to sleep and shifts to another task
 - After finishing the task, the device raises an interrupt
 - The ISR is called
 - The OS reads the result of the I/O, wakes the sleeping process



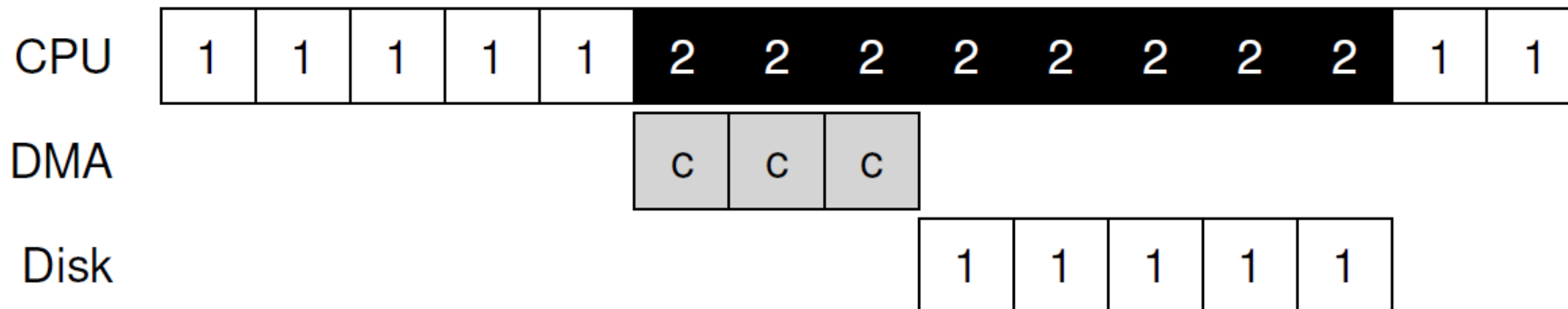
- For faster devices, Polling might be better than interrupts!
- Also, in networks, if each incoming packet raises an interrupt and there are too many packets coming in!
- Coalescing helps, sometimes.

Direct Memory Access (DMA)

- Programmed I/O is also very expensive!



- DMA engine can transfer data b/w RAM and device w/o CPU intervention



Communicating with devices

- Two main techniques:
 - Explicit I/O instructions
 - Intel has in and out instructions
 - Memory mapped I/O

OS is oblivious to device details

- Each device has its own characteristics
- The File System should work on all of them nonetheless
- How do we do it?
- Abstraction!
- Device driver is a layer that hides the details

